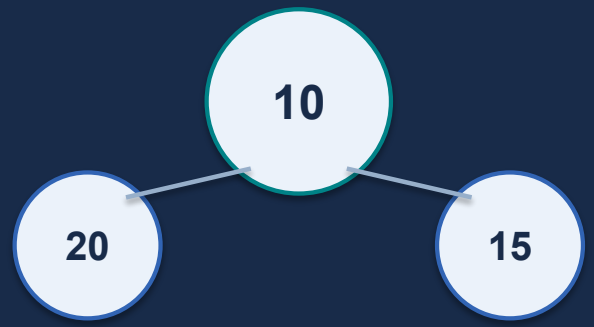
DATA STRUCTURES

Binary Heaps & Priority Queues



Learning outcomes

By the end of this lecture, you should be able to…

- Define a binary heap and explain the complete binary tree property.
- Differentiate between Min Heap and Max Heap.
- Represent a heap using a zero-indexed array.
- Trace insertion and root deletion step by step.
- Explain how heaps support priority queues and analyze operation costs.

Quick recap: binary trees

Connect the new topic to the trees you already know.

Binary tree

Each node has at most two children.

Binary Search Tree

Left subtree values are smaller; right subtree values are larger (under the usual distinct-key rule).

Today's question

How can we quickly access the smallest or largest item?

A heap solves a different problem from a BST: it keeps the minimum or maximum at the root.

Why do we need a heap?

Think about systems that repeatedly choose the most urgent or important item.

Hospital triage

Treat the most urgent case first.

Task scheduler

Run the task with the highest priority.

Event simulation

Process the next event with the earliest time.

A priority queue is the abstract data type; a binary heap is a common way to implement it.

What is a Binary Heap?

A binary heap must satisfy two properties.

1. Shape property

It is a complete binary tree: every level is full except possibly the last, and the last level fills from left to right.

2. Heap-order property

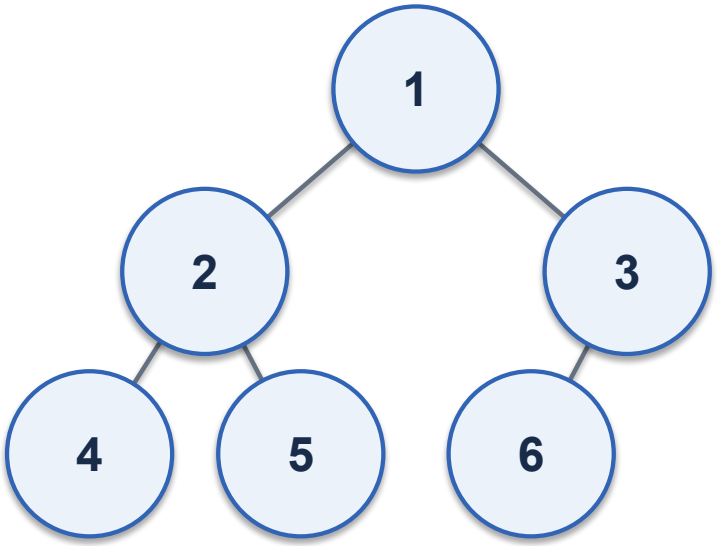
Min Heap: parent ≤ children. Max Heap: parent ≥ children.

Both properties must hold at the same time.

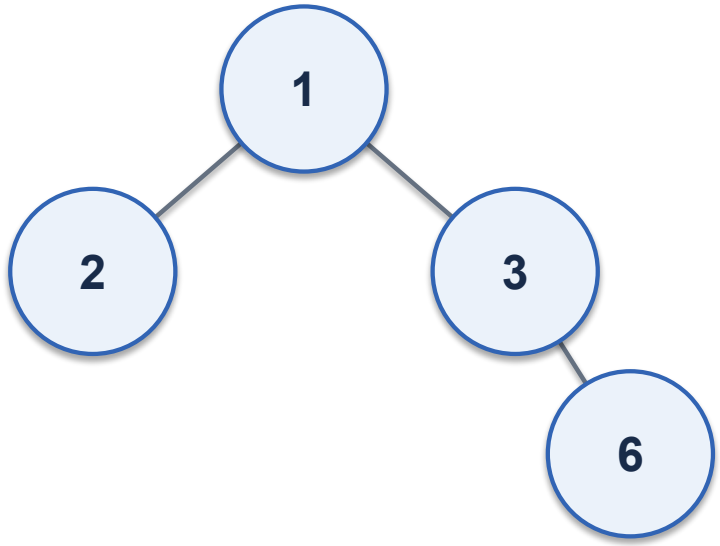
Complete binary tree: shape matters

The final level fills from left to right.

Complete



Not complete

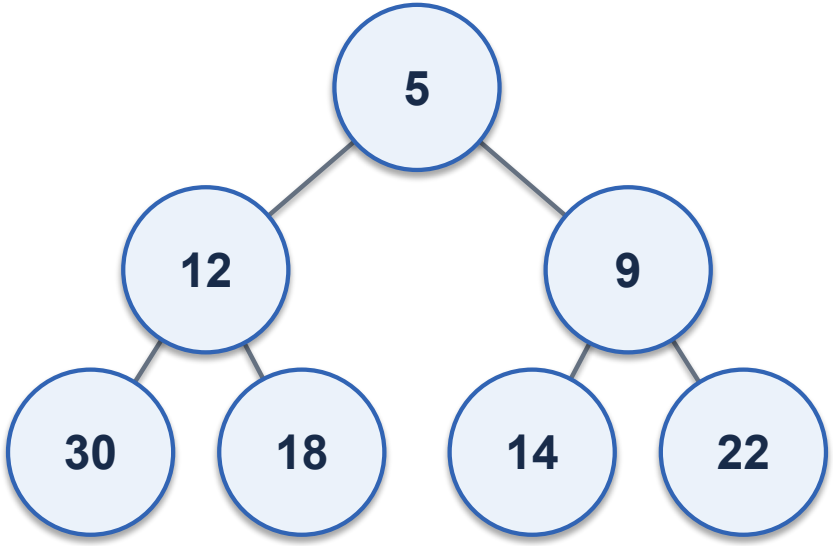


A gap appears before a later node.

Min Heap vs Max Heap

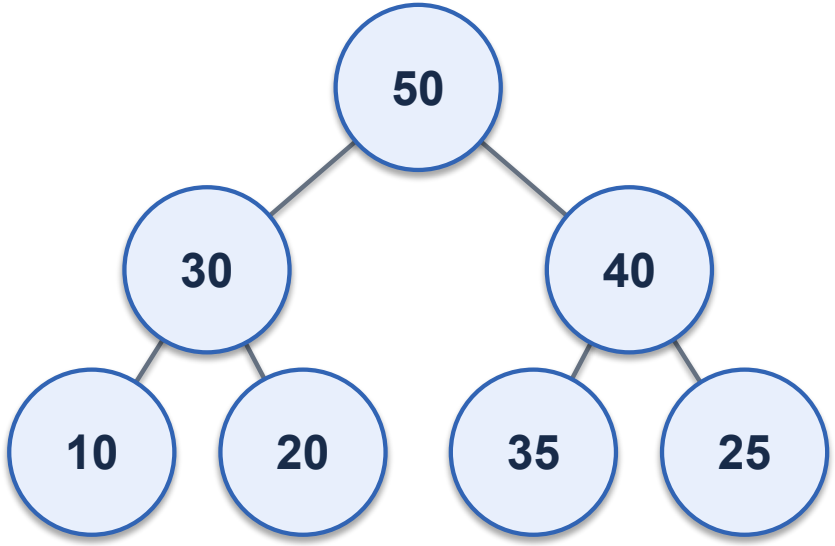
The root is the most important element for the heap's ordering rule.

MIN HEAP



Smallest value at the root

MAX HEAP

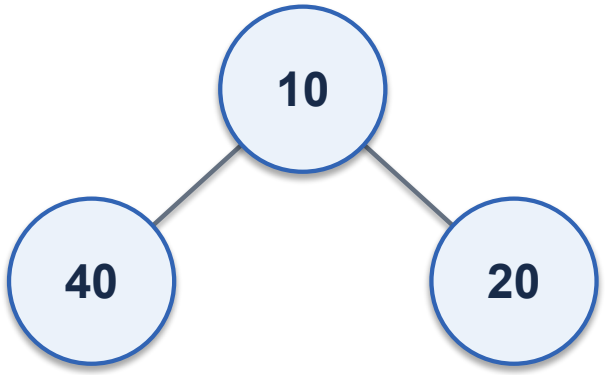


Largest value at the root

Important: a heap is NOT a BST

Heap ordering only compares each parent with its children.

This is a valid Min Heap



Why?

$10 \leq 40$ and $10 \leq 20$.

The heap rule is satisfied even though the left child (40) is greater than the right child (20).

Do not apply BST search logic to a heap.

Array representation (zero-indexed)

A complete binary tree can be stored compactly without child pointers.



- Parent of index i: (i − 1) / 2 (integer division; root has no parent)
- Left child of index i: 2i + 1
- Right child of index i: 2i + 2

Example: for index 1, left child index = 3 and right child index = 4.

Let's calculate the indices

Use the array from the previous slide.



Index 0: value 10

Left child: index 1

Right child: index 2

Index 1: value 20

Left child: index 3

Right child: index 4

Index 2: value 15

Left child: index 5

Right child: index 6

The formulas work because the tree is complete.

Insertion in a Min Heap

Two stages: place at the end, then restore heap order.

Stage 1: Insert at end

Add the new value in the next available array position to preserve completeness.

Stage 2: Heapify up

Compare with the parent. If the new value is smaller, swap and repeat until valid.

Insertion takes O(log n) time in a binary heap.

Insertion dry run: insert 8

Start with this valid Min Heap.



Insert value: 8

- Place 8 at the end (index 7).
- Compare 8 with parent 30 at index 3.
- Swap because 8 < 30.
- Compare 8 with parent 20; swap.
- Compare 8 with root 10; swap.

Array after placing 8

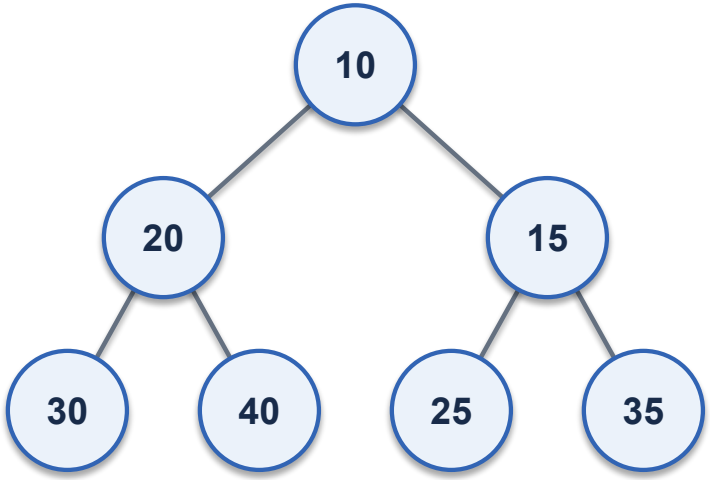


After heapify up: [8, 10, 15, 20, 40, 25, 35, 30]

Insertion: follow the swaps

Heapify up continues until the parent is no larger than the inserted value.

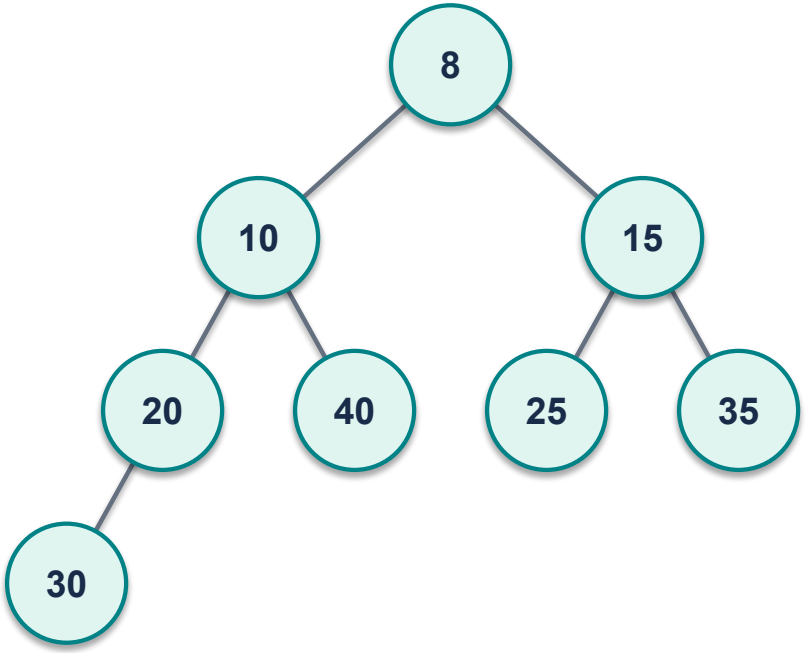
Before insertion



→ 8 added

then swapped ↑

After heapify up



Root is now 8; heap property restored.

Delete the root of a Min Heap

Deletion usually removes the minimum value (the root).

- Save/remove the root value.
- Move the last element to the root and shrink the heap.
- Heapify down: compare the parent with its children.
- Swap with the smaller child when the parent is too large.
- Repeat until the Min Heap property is restored.

For a Max Heap, root deletion removes the maximum and swaps with the larger child.

Deletion dry run: remove the minimum

Initial Min Heap array: [5, 12, 9, 30, 18, 14, 22]



Step 1: Move last value (22) to root



Step 2: Compare children 12 and 9

Smaller child = 9

Swap 22 and 9

Now 22 is compared with its children 14 and none; swap with 14.



Final heap array: [9, 12, 14, 30, 18, 22]

Priority Queue: the connection

A priority queue removes items by priority, not simply by arrival order.

Min-priority queue

The smallest priority value is removed first.

Example: earliest event time.

Max-priority queue

The largest priority value is removed first.

Example: highest-priority task.

A binary heap gives efficient access to the highest-priority item.

Note: “priority” can be represented by a number; decide whether smaller or larger means more urgent.

Heap operations and time complexity

Typical costs for a binary heap stored in an array.

Operation	Time	Reason
Peek root (min/max)	**O(1)**	Root is at index 0
Insert	**O(log n)**	Heapify up
Delete root	**O(log n)**	Heapify down
Search for an arbitrary value	**O(n)**	Heap does not provide BST ordering
Build heap from n items	**O(n)**	Bottom-up heap construction

Common mistakes to avoid

These are frequent sources of incorrect dry runs.

- Assuming a heap is sorted from left to right.
- Confusing the heap property with the BST property.
- Forgetting to preserve the complete-tree shape during insertion.
- During Min Heap deletion, swapping with the larger child instead of the smaller child.
- Stopping heapify before the ordering property is restored.

Always check BOTH the shape property and the heap-order property.

Guided practice

Work through these on paper before checking with a partner.

Task A: insertion

Insert 7 into the Min Heap:

[10, 18, 12, 25, 30, 20]

Task B: deletion

Delete the root from:

[4, 11, 6, 20, 15, 9]

For each task, show the array after every swap and explain why the swap is required.

Extension: Which operation is O(1), and why is arbitrary search O(n)?

Practice check: answer key

Compare your intermediate steps, not just the final arrays.

Task A: insert 7

Start: [10, 18, 12, 25, 30, 20]

Append 7 → [10, 18, 12, 25, 30, 20, 7]

Swap with 12, then with 10.

Final: [7, 18, 10, 25, 30, 20, 12]

Task B: delete root

Start: [4, 11, 6, 20, 15, 9]

Move 9 to root → [9, 11, 6, 20, 15]

Swap with smaller child 6.

Final: [6, 11, 9, 20, 15]

Root peek is O(1); searching an arbitrary value can require checking all elements: O(n).

Lecture summary

Keep these five takeaways in your notes.

- A binary heap is a complete binary tree.
- Min Heap: parent ≤ children; Max Heap: parent ≥ children.
- A zero-indexed array stores heap nodes efficiently.
- Insertion uses heapify up; root deletion uses heapify down.
- Priority queues can be implemented efficiently with binary heaps.

Next lecture: C++ implementation of heap insertion and deletion.

Exit ticket

Answer in 2–3 sentences before leaving.

1. What are the two properties of a binary heap?

2. In a Min Heap, which child should replace a parent during heapify down?

3. Why is searching for an arbitrary value O(n)?

Be ready to explain each answer without looking at the slides.